
Autonomous Systems and Mobile Robots

Exercise Sheet 3 – Summer Term 2026

Issued: 12.05.2026

Professor

Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor

Janosch Bajorath
j.bajorath@uni-muenster.de

Your robot from Week 2 can drive and report its own position via odometry — but it has no model of the world around it. *Odometry is proprioceptive*: the robot measures its own internal state (wheel rotations) and infers where it *thinks* it is. It has no way to check what is in front of it.

Today you add an *exteroceptive* sensor: a 2D LiDAR. A LiDAR fires a rotating laser, measures the return time for each angle, and produces a ring of distances — one value per degree, published as a `sensor_msgs/msg/LaserScan` message on the `/scan` topic. The robot can now observe the environment it moves through.

What you will do: add a LiDAR link and Gazebo sensor plugin to the Week 2 URDF, bridge `/scan` into ROS 2, visualise the scan in RViz, inspect the message structure, and finally compare what odometry and LiDAR each say about the same motion.

What carries over: the three packages from Week 2 are extended in place — no new workspace or package is created. In Week 4 and 5 (MiniLab 1) you will write a node that reads `/scan` and publishes `/cmd_vel`: that closes the Perceive–Reason–Act loop.

Task 1 – Add the LiDAR to the URDF

A Gazebo sensor plugin simulates the physics of a sensor and publishes data on a Gazebo-internal topic. In this task you extend your robot description with a LiDAR link and the corresponding sensor plugin.

API. Parameter meanings and accepted values are in the Gazebo sensor documentation: <https://gazebo-sim.org/docs/harmonic/sensors/>. The correct sensor type for Gazebo Harmonic is `gpu_lidar`. The Gazebo-side topic name (`lidar`) is *not* the ROS2 topic name — the bridge in Task 2 maps one to the other.

(a) Add the sensor link and joint.

In `my_robot_description/urdf/my_robot.urdf.xacro`, add:

- a `lidar_link` — a small cylinder (radius 0.05 m, length 0.04 m) representing the sensor housing. Give it a mass of 0.1 kg and reasonable inertia.
- a `lidar_joint` of type `fixed`, with parent `base_link` and child `lidar_link`. Place the sensor on top of the chassis, centred: `origin xyz="0 0 0.07"`.

Formula. Cylinder inertia: $I_{xx} = I_{yy} = \frac{m}{12}(3r^2 + l^2)$, $I_{zz} = \frac{m}{2}r^2$. For $m = 0.1$, $r = 0.05$, $l = 0.04$: roughly $I_{xx} \approx 7.6 \times 10^{-5}$, $I_{zz} \approx 1.25 \times 10^{-4}$.

(b) Add the Gazebo sensor plugin.

In `robot.gazebo`, add a `<gazebo reference="lidar_link">` block containing a `<sensor type="gpu_lidar">` element. Derive the full plugin structure from the official Gazebo sensor documentation:

<https://gazebo-sim.org/docs/harmonic/sensors/>

Configure the sensor with:

- 360 horizontal samples, 1-degree resolution, sweeping $-\pi$ to π
- range: 0.1 m to 10.0 m, resolution 0.01 m
- Gazebo topic name: `lidar`
- `gz_frame_id`: set to `lidar_link` — this ensures the published scan message carries the correct TF frame even if the fixed joint is collapsed during URDF-to-SDF conversion
- update rate: 10 Hz
- `<always_on>true</always_on>` and `<visualize>true</visualize>`

Note. The snippet also contains a pre-filled `<vertical>` block (1 sample, angles 0 to 0). Leave it as-is — a 2D LiDAR sweeps a single horizontal plane, but Gazebo Harmonic requires the vertical section to be present for the sensor to initialise correctly.

Syntax. Open `snippets/lidar_sensor.gazebo` alongside this task. It shows the complete `<gazebo reference="lidar_link">` block — paste it into `robot.gazebo` as a new top-level element, alongside the existing `<gazebo>` blocks, and fill in the TODO values listed above. Do *not* nest it inside another `<gazebo reference="lidar_link">` element.

(c) **Enable the Sensors plugin in your world file.**

The `gpu_lidar` sensor requires Gazebo's Sensors system to be loaded — without it the sensor is configured but never activated, and `/scan` will not appear. Open `my_robot_bringup/worlds/empty_world.sdf` and add the following plugin directly after the `SceneBroadcaster` entry. The plugin must be a direct child of `<world>` — not nested inside any `<model>` or other element.

```
<plugin filename="gz-sim-sensors-system"
  name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
```

(d) **Rebuild and verify.**

Run `colcon build`, source the install overlay, and launch `robot_vis.launch.py`. Confirm that:

- the RViz robot model shows a new cylinder on top of the chassis
- `ros2 run tf2_tools view_frames` includes `lidar_link` in the TF tree, as a child of `base_link`

Then launch `robot_sim.launch.py` and confirm:

- `ros2 topic echo /scan --once` prints a `LaserScan` message with `frame_id: lidar_link` and a `ranges` array of 360 values

(e) **Discussion.**

Why does the placement of the LiDAR's coordinate frame matter for interpreting scan data? What would change if the sensor were mounted off-centre or tilted?

Task 2 – Bridge /scan

The bridge (`ros_gz_bridge`) connects two separate message buses: Gazebo's internal gz-transport bus and the ROS2 DDS bus. Neither side can see the other's topics without a bridge entry. Your Week 2 `bridge.yaml` already maps five topics — `/cmd_vel`, `/odom`, `/clock`, `/joint_states`, and `/tf`. Each entry uses the same five fields: `ros_topic_name`, `gz_topic_name`, `ros_type_name`, `gz_type_name`, and `direction`. Today you add a sixth entry for `/scan`.

API. To find the correct type pair for any Gazebo topic, use two approaches.

At runtime (once Gazebo is running):

```
gz topic -l          # list all active Gazebo topics
gz topic -i -t /topic_name # show the type of a specific topic
```

Reference table: the `ros_gz_bridge` README at https://github.com/gazebo-sim/ros_gz contains the full mapping of all supported type pairs.

For `/scan` the pair is `sensor_msgs/msg/LaserScan` ↔ `gz.msgs.LaserScan`.

(a) **Add the bridge entry.**

Open `my_robot_bringup/config/bridge.yaml` and add an entry for `/scan`.

Hint. Your existing `bridge.yaml` already contains entries for `/cmd_vel`, `/odom`, `/clock`, `/joint_states`, and `/tf` — use them as a structural reference. The `gz_topic_name` must be prefixed with `/:` if your plugin sets `<topic>lidar</topic>`, the bridge entry needs `gz_topic_name: /lidar`.

(b) **Launch and verify.**

Rebuild, source, and launch `robot_sim.launch.py`. Confirm:

- `ros2 topic list` includes `/scan`
- `ros2 topic hz /scan` reports approximately 10 Hz

Task 3 – Visualise in RViz

RViz's LaserScan display subscribes to `/scan` and renders each distance measurement as a coloured point in 3D space. It uses `angle_min` and `angle_increment` from the message header to compute the direction of each beam, then transforms the resulting points into the Fixed Frame using the TF tree. Setting the Fixed Frame to `odom` anchors the scan to the world — you can observe where obstacles are as the robot moves, rather than seeing the scan frozen relative to the robot body.

(a) **Add a LaserScan display.**

In RViz, click *Add* and choose *LaserScan*. Set the topic to `/scan`. Set the *Fixed Frame* to `odom`. Increase *Size (m)* to 0.05 so individual scan points are visible. Drive the robot toward a wall in Gazebo and observe the scan update in real time.

(b) **Save the RViz config.**

Overwrite the existing `.rviz` config file in `my_robot_bringup/config/` so the LaserScan display persists across launches.

(c) **Inspect the computation graph.**

Run `rqt_graph`. Identify:

- which node publishes `/scan`
- which nodes (if any) currently subscribe to it

(d) **Switch the Fixed Frame.**

In RViz's *Global Options*, change the Fixed Frame successively to `lidar_link`, `base_link`, and `odom`. Observe how the scan repositions each time.

Concept. RViz applies the TF transform automatically when you change the Fixed Frame — the scan data does not change, only the coordinate system it is displayed in. In Task 6 you will perform this same operation explicitly in Python code.

Task 4 – Inspect the Scan Message

The `ranges` array is the core of every `LaserScan` message: one distance per beam, ordered from `angle_min` to `angle_max` at equal `angle_increment` steps. Because the grid is uniform, any bearing can be addressed with a single formula. This is the same formula your MiniLab 1 reactive node will use to read the distance straight ahead — internalising it now saves debugging time later.

- (a) **Echo one message.**

Run:

```
1 ros2 topic echo /scan --once
```

Read off and record: `angle_min`, `angle_max`, `angle_increment`, `range_min`, `range_max`, and the length of the `ranges` array.

- (b) **Find the straight-ahead index.**

Using `angle_min` and `angle_increment`, calculate which index in `ranges` corresponds to a bearing of 0rad — i.e. directly in front of the robot. Show your calculation.

- (c) **Plot live.**

Open `rqt_plot` and enter the straight-ahead range as the topic expression.

Convention. To plot the element at the index you calculated in (b), enter `/scan/ranges[N]` in the topic field, where N is your computed index.

Drive the robot toward a wall and observe the plotted value decrease. Drive away and observe it increase. Use the following command to drive with the keyboard:

```
1 ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Concept. This works without any configuration because your robot follows ROS2 conventions: `teleop_twist_keyboard` publishes `geometry_msgs/msg/Twist` on `/cmd_vel`, which is exactly what your `diff_drive` controller subscribes to. Any node that speaks the same message type on the same topic integrates automatically — this is the payoff of following standard interfaces.

What happens when the reading exceeds `range_max`?

- (d) **Discussion.**

The `ranges` array is a fixed-length snapshot of the scan at one instant. What would happen to the forward-facing index if you changed the angular resolution from 1 degree to 0.5 degrees?

Task 5 – Odometry vs. LiDAR

Odometry and LiDAR both provide information about the robot's position relative to its environment — but they measure fundamentally different things. In this task you set up a controlled scenario and compare what each sensor reports about the same motion.

(a) **Set up.**

Copy `snippets/wall_world.sdf` to `my_robot_bringup/worlds/wall_world.sdf`. Then open `robot_sim.launch.py` and change `'empty_world.sdf'` to `'wall_world.sdf'` in the `world_path` variable.

Rebuild, launch `robot_sim.launch.py`, and confirm the wall is visible in Gazebo.

Before driving, record:

- the straight-ahead `ranges` value (index from Task 4)
- the `pose.pose.position.x` field from `ros2 topic echo /odom --once`

Convention. Once you have confirmed the world looks correct, you can run Gazebo *headless* (no GUI) to save compute resources. Add `-s` to the `gz_args` string in `robot_sim.launch.py`:

```
gz_args='-r -s world_path'
```

The `-r` flag starts the simulation running immediately; `-s` disables the Gazebo GUI. RViz and all ROS 2 topics continue to work normally.

(b) **Drive forward.**

Publish a constant forward velocity for exactly 5 s to move the robot 1 m (at 0.2 m/s):

```
1 ros2 topic pub --rate 10 --times 50 /cmd_vel geometry_msgs/msg/Twist \
2   "{linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

The command stops automatically after 50 messages (5 s). Watch `/odom` in `rqt_plot` (`/odom/pose/pose/position/x`) to confirm the displacement.

(c) **Compare.**

Record the new odometry position and the new straight-ahead `ranges` value. Fill in the table:

Measurement	Expected	Measured
/odom displacement (m)	1.0	
LiDAR distance change (m)	1.0	
Final LiDAR distance to wall (m)	1.0	

Do the values agree?

(d) **Reflect.**

In simulation the values should match closely. On a real robot they often do not.

- Name one scenario in which *odometry* is more reliable than LiDAR.
- Name one scenario in which *LiDAR* is more reliable than odometry.

Task 6 – Where Is That Obstacle?

Raw `ranges` values tell you the distance to a surface along a single beam — useful for threshold reactions, but expressed in the sensor’s own frame. The moment you ask a more geometric question — *will my robot body actually hit that?* — you need the obstacle’s position in `base_link`, the frame centred on the robot body, so you can compare it against the robot’s physical footprint.

In this task you write a ROS 2 node that finds the nearest obstacle in the scan, transforms its position from `lidar_link` to `base_link` using the TF tree, and logs both coordinates side by side. This is the operation RViz performed automatically in Task 3d — here you do it explicitly in Python.

API. `tf_buffer.lookup_transform(target_frame, source_frame, time)` returns a `TransformStamped`. Pass `rclpy.time.Time()` as the time argument to request the latest available transform — matching the exact scan timestamp can fail if TF lags slightly behind the sensor.

`tf2_geometry_msgs.do_transform_point(point_stamped, transform)` returns a new `PointStamped` in the target frame. The `import tf2_geometry_msgs` line must be present even if the module itself is never called directly — it registers the transform handler for `PointStamped` at import time.

(a) From polar to Cartesian.

A LiDAR beam at index i carries a distance $r = \text{ranges}[i]$ and an angle $\theta = \text{angle_min} + i \times \text{angle_increment}$. The corresponding point in the `lidar_link` frame is:

Formula.

$$x = r \cos \theta, \quad y = r \sin \theta, \quad z = 0$$

$z = 0$ because a 2D LiDAR sweeps a horizontal plane; all measurements lie in the sensor’s xy -plane.

1. For the nearest obstacle (the beam with the minimum finite range value), calculate θ and the Cartesian point (x, y) by hand using the values you read in Task 4.
2. What frame is this point expressed in?
3. What coordinate change do you expect after transforming to `base_link`? (Hint: check the `lidar_joint` origin in your URDF.)

(b) **Implement the node.**

Copy `snippets/scan_to_base.py` into your package:

```
1 cp snippets/scan_to_base.py \  
2   src/my_robot_control/my_robot_control/scan_to_base.py
```

Then complete the six TODO blocks.

Convention. Register the entry point in `setup.py` under `console_scripts`:

```
'scan_to_base = my_robot_control.scan_to_base:main',
```

Rebuild with `colcon build --symlink-install` and source the overlay before running.

(c) **Verify.**

With `robot_sim.launch.py` running, start the node in a second terminal:

```
1 ros2 run my_robot_control scan_to_base
```

Confirm:

- The log output shows two coordinate triples per scan.
- The `base_link` z value is approximately $+0.07\text{m}$ — the sensor mount height.
- Drive the robot toward a wall: the `base_link` x value decreases as the robot approaches.